



**ENT-164** · Intro to Making



Derby Entrepreneurship  
Center at Tufts

CLASS 5 · WEEK 5 · FALL 2026

# Electronics

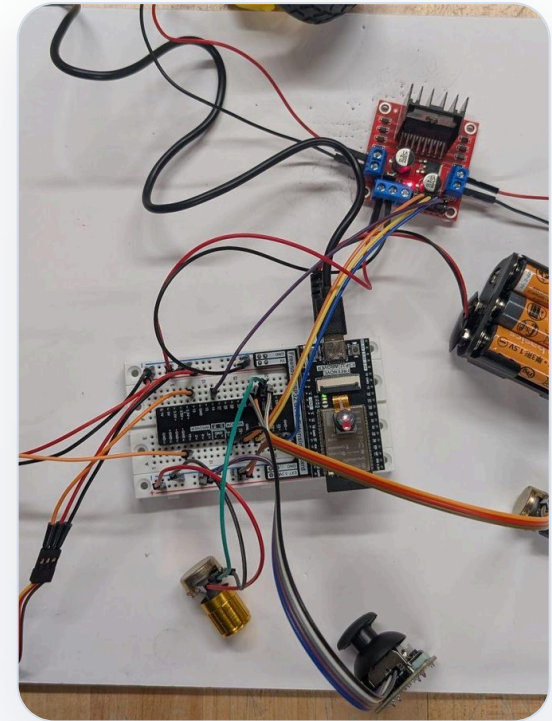
Today your robot gets a brain. We wire motors, sensors, and controls on the breadboard, install MicroPython, and make AI our coding partner.

Thu, Oct 8

JCC 301 / Nolop

Robo Challenge

Bring your ESP32 kit



# Seven weeks from sketch to race day

Today is Week 5. Basic Electronics and Microcontrollers. Here's how the robot build comes together.

## Week 2

### Ideation and Prototyping

Sketch concept variations; build a foam core mockup.

## Week 3

### From Physical to Digital: 2D

Create a laser-ready 2D design; cut structural parts.

## Week 4

### From Physical to Digital: 3D

Model 3D parts; 3D print functional parts.

## Week 5

### Basic Electronics and Microcontrollers

Wire motors, sensors, and controls; program with MicroPython + AI.

## Week 6

### AI Coding Agents

Connect to a wireless network; build a remote control interface.

## Week 7

### Final Robot Assembly and Testing

Final integration and test runs.

## Week 8

### Robo Challenge and Final Project Kickoff

Demonstrate the robot with mechanical systems and motors mounted.

## TODAY

**Week 5 · Electronics.** Wire motors, sensors, and controls on a breadboard, and program basic movements using MicroPython and AI.

# Today the robot gets a brain

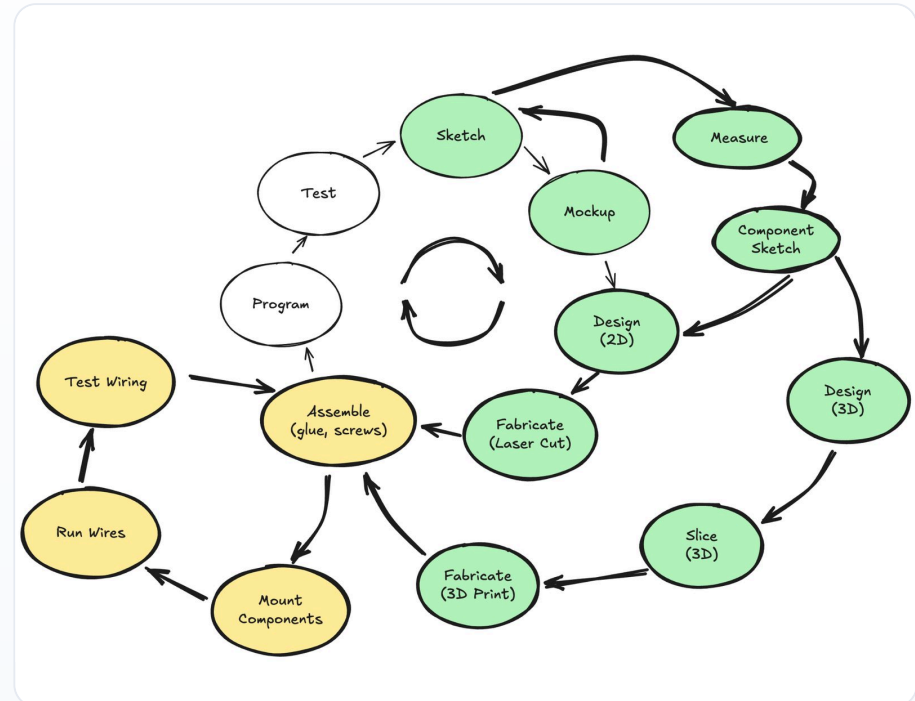
The mechanics are nearly done. Today we bring the electronics to life.

## TODAY'S PLAN

- Meet your kit and the ESP32 board
- Install the MicroPython operating system
- Blink an LED with opencode
- Move a servo with AI-generated code
- Wire the robot at Nolop

## WHERE WE ARE IN THE BUILD

Sketch, mockup, design, and fabrication are behind you. Now we assemble, run wires, test the wiring, program, and test again.



# What's in your **personal kit**?

Freenove FNK0046 Super Starter Kit for ESP32 WROVER. Every part the electronics project needs.

## The brain

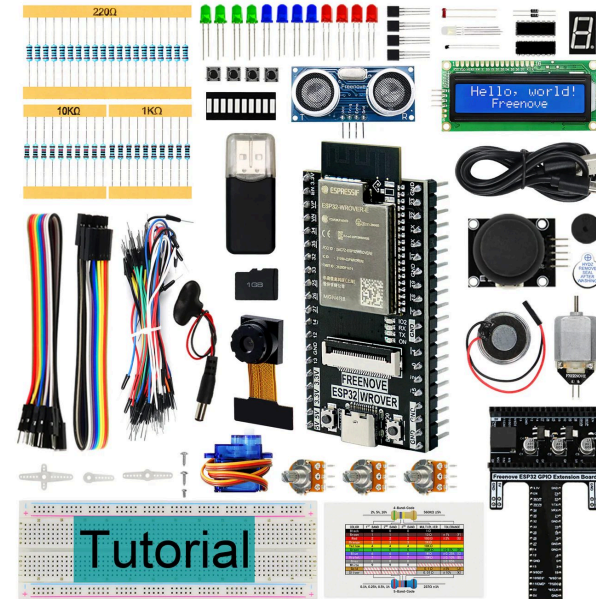
ESP32-WROVER board, GPIO extension board, and a solderless breadboard.

## The basics

Jumper wires, LEDs, resistors (220 $\Omega$ , 1K, 10K), push buttons, USB cable.

## The fun stuff

LCD and 7-segment displays, buzzers, joystick, potentiometers, servo, TT motor + wheel, motor driver, ultrasonic sensor, camera.



### NOTE

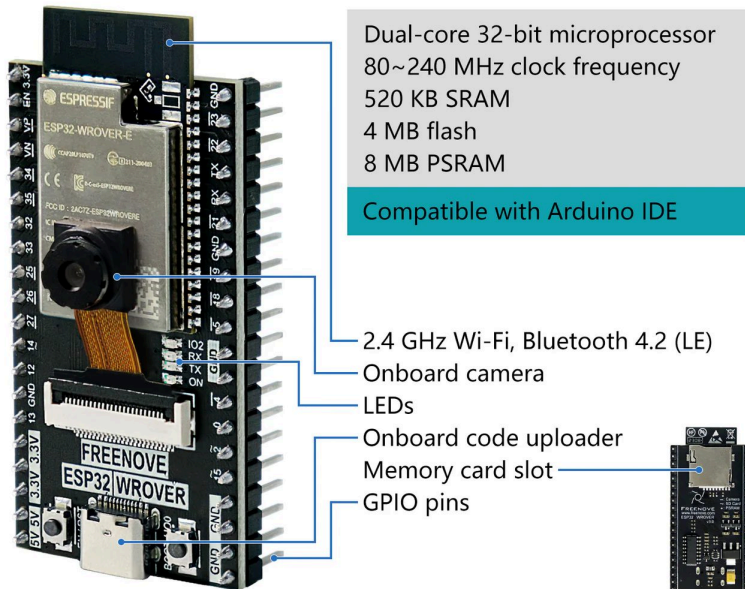
No paper tutorial. The download link for the tutorial and code is **on the box**.

# Meet the ESP32-WROVER

A powerful, small controller with onboard wireless and camera.

## ESP32-WROVER Board

A powerful, generic and small-sized controller with onboard wireless and camera.



### SPECS

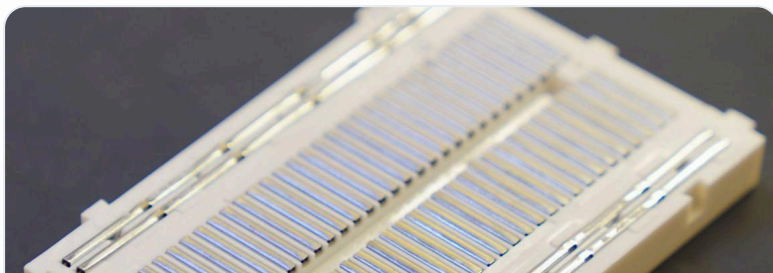
- Dual-core 32-bit microprocessor, 80–240 MHz
- 520 KB SRAM · 8 MB PSRAM · 4 MB flash
- 2.4 GHz Wi-Fi and Bluetooth 4.2 LE
- Onboard camera, LEDs, and code uploader
- Memory card slot and GPIO pins

### WHAT IT CONTROLS

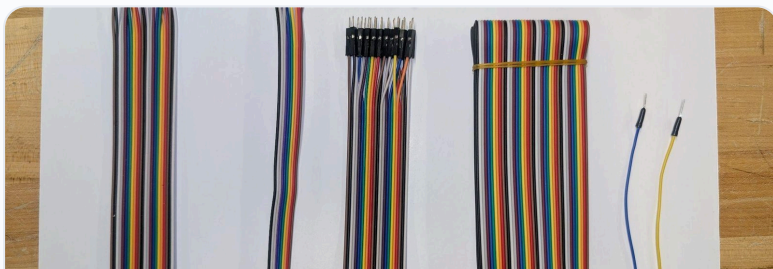
Motors through the driver, servos, joystick, potentiometers, sensors, and displays. The brain of your robot.

# How the **breadboard** works

No soldering. Push parts and wires into the holes and they connect.



A solderless breadboard. Rows of connected holes



Jumper wires. Male–male, male–female, female–female

## Rows

Each row of five holes is connected horizontally, and the center channel separates the two sides.

## Power rails

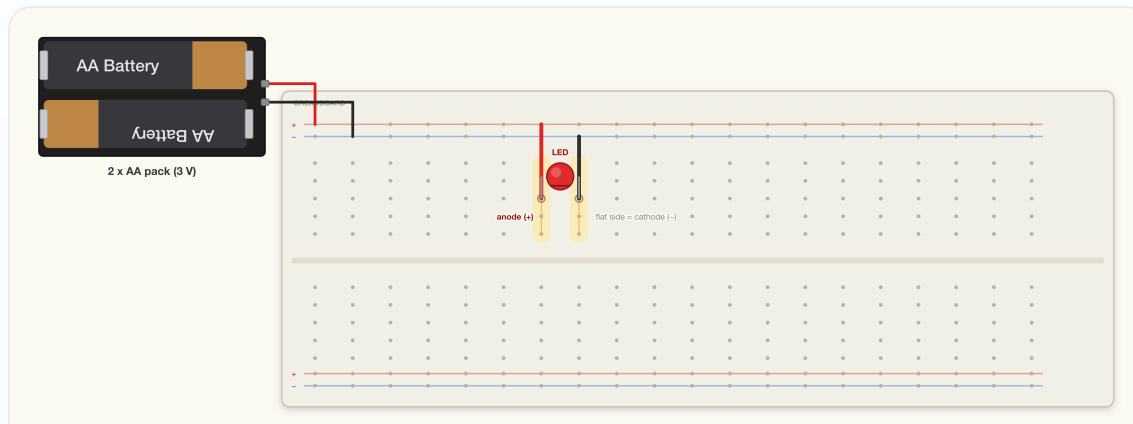
The vertical rails along the edges carry 5V, 3.3V, and GND to every row.

## Pin rows

Your ESP32 plugs into the GPIO extension board: any of the horizontal holes in a pin's row connects to that pin. For example, pin 34.

# An LED with no resistor

The LED straight across the rails. It lights, it is very bright, and it is being overdriven.



## Follow the steps

The diagram lists them, and the class page has the same one at full size.

## Wire it

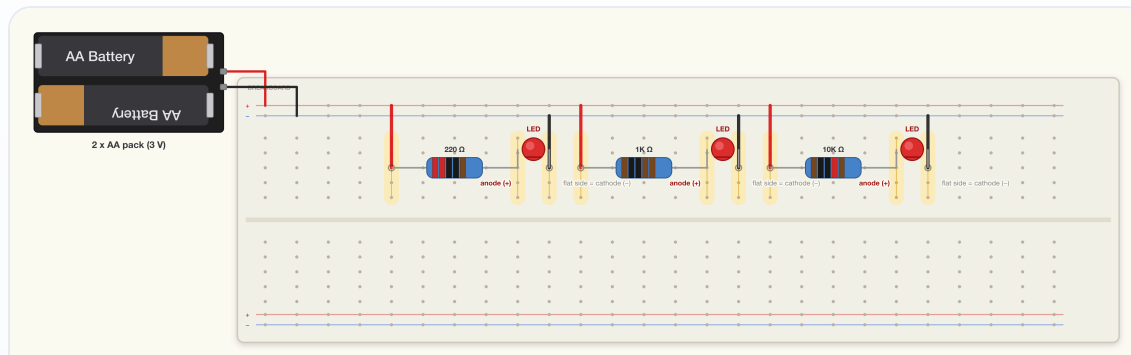
- Pack at the left: red lead to the + rail, black to the -
- LED: long leg in column 6, short leg in column 7, row c
- Red jumper: the + rail to column 6. Black jumper: column 7 to the - rail

## On for a second

Connect it, look, then pull the red lead back out. Then take it apart and build exercise 2.

# 220 ohm, 1 kohm, 10 kohm

The same LED with three different resistors. Bright, dimmer, barely on.



## Follow the steps

Three branches in one build, so you can compare them side by side.

## Wire it

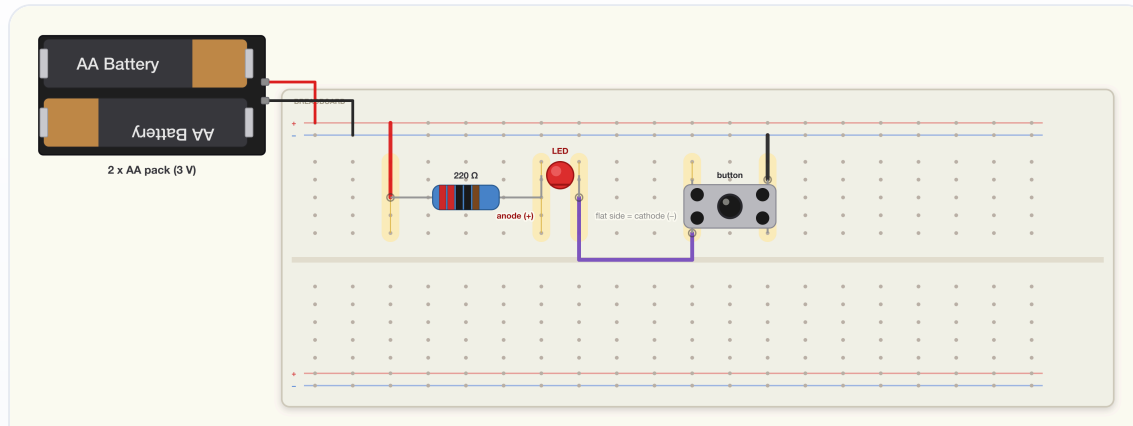
- A resistor and LED per branch: 220  $\Omega$ , then 1K  $\Omega$ , then 10K  $\Omega$ , row c
- Red jumpers: the + rail to each resistor's left leg
- Black jumpers: each LED back to the - rail

## More resistance, less current

The resistor decides how much current the LED gets, and the current decides how bright it is.

# Add a push button

The 220  $\Omega$  circuit with a switch in the path. Press to light the LED, let go and it goes dark.



## Follow the steps

The button goes in the path, not beside it.

## Wire it

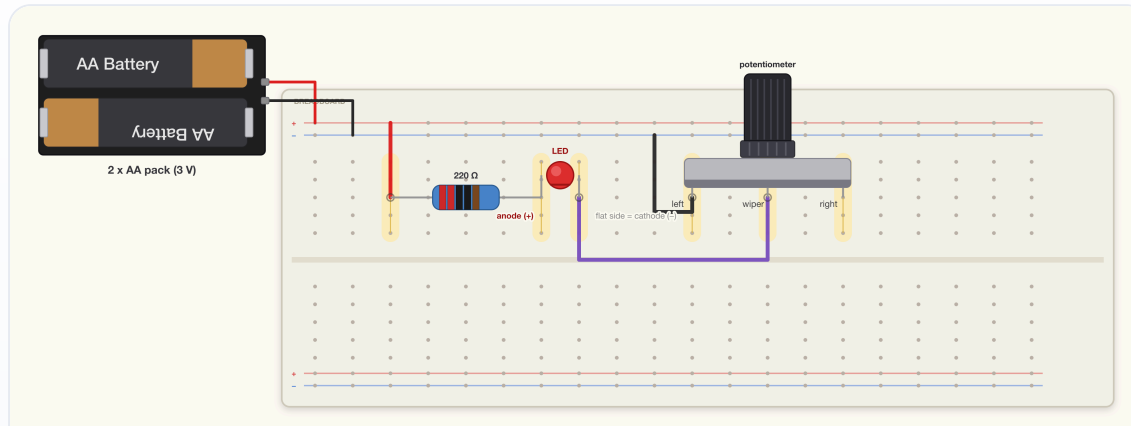
- Resistor: the + rail to column 2, body to column 6
- LED: columns 6 and 7, row c
- Button: columns 10 and 12, rows b and e
- Purple jumper: column 7 to column 10. Black: column 12 to the - rail

## Two states, nothing between

Pressed or not. On or off. That is a digital signal, and a pin on the board reads it the same way.

# Swap in a potentiometer

Take the button out and put the knob in its place. Turning it makes the LED brighter and dimmer.



## Follow the steps

One end of the track takes the current, the wiper passes it on.

## Swap it

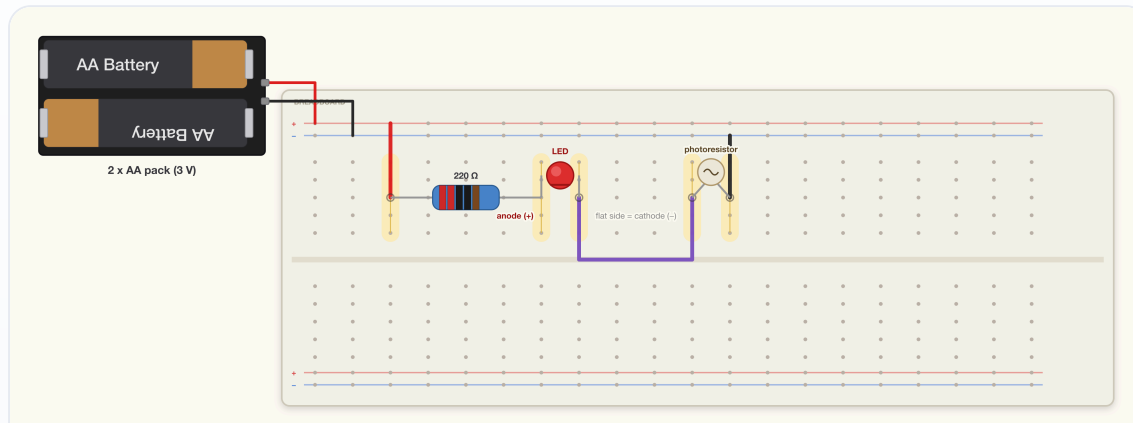
- Take the button out
- Pot: legs in columns 10, 12 and 14, row c
- Purple jumper: column 7 (the LED) to column 12, the middle leg
- Black jumper: column 10, the end of the track, to the - rail

## A range, not two states

The wiper taps the track wherever you turn it, so the resistance changes smoothly. That is analogue.

# Swap in a photoresistor

Take the knob out and put a light sensor in its place. The LED follows the light.



## Follow the steps

The sensor takes the knob's place in the same path.

## Swap it

- Take the pot out
- Photoresistor: its two legs in columns 10 and 11, row c
- Purple jumper: column 7 to column 10
- Black jumper: column 11 to the - rail

## Light is the knob

More light, less resistance, more current, brighter LED. Cover it and the LED fades.

# Signals **travel on the wire**

A voltage, held or changed, read at the other end. That is the whole conversation between a microcontroller and a peripheral.

01

## Digital

Two states: 3.3 V or 0 V. Your button. A pin reads it, and writes it.

02

## Analogue

A voltage anywhere in the range. Your knob and your light sensor. An ADC pin turns it into a number.

03

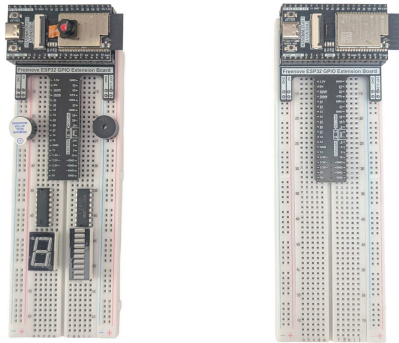
## Patterns

Switch a pin fast, and the average is what matters: that is how brightness and servo angles are sent. Modules that pass numbers do the same, in a pattern both ends agree on.

**Next: the ESP32 does this in code.** The same LED, the same button, the same knob, with a pin doing the reading and the writing.

# Prep your board

Three steps before the first circuit.



1 · Remove the pre-installed demo parts

02

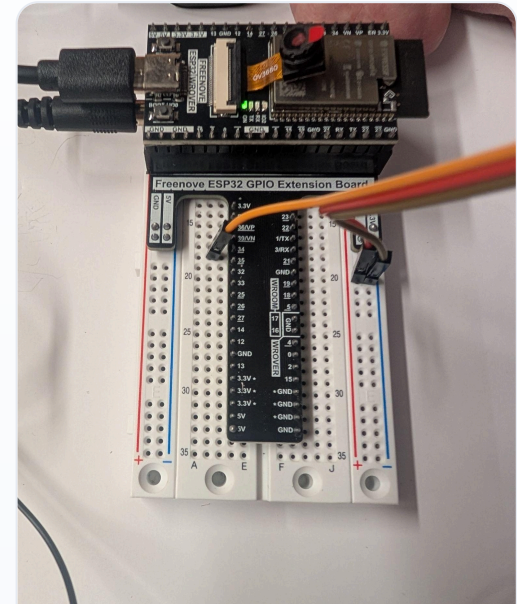
## Seat the board

Clip the ESP32 and its GPIO extension board across the breadboard's center channel.

03

## Read the labels

5V, 3V3, GND, and the numbered GPIO pins are printed on the extension board. Check before you plug anything in.



Pin labels on the extension board

# Install opencode with the **setup guide**

Join the class workspace, install the app, add the class skills. One guide, macOS and Windows.

## The setup guide

`tuftsmaker.github.io/ENT-164/opencode-setup`

## It walks through

- Accept the class invitation, then sign in
- Install the app and choose the class model
- Add the class skills, then ask it to flash the board



The MicroPython snake needs a plane. Your ESP32 board.

# How it works

You asked opencode to blink an LED. Here's what happened behind the scenes.

## BLINK.PY

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)

while True:
    led.on()      # Turn LED on
    time.sleep(1) # Wait 1 second
    led.off()     # Turn LED off
    time.sleep(1) # Wait 1 second
```

## WHAT THE AGENT DID

- Wrote MicroPython for the onboard LED on GPIO 2
- Put the MicroPython runtime on the board, then copied the program in over USB
- Started it — the LED blinks the moment the program lands

## LOOK INSIDE

Ask opencode: “Show me the code you just wrote and explain it line by line.”

Then push further: what is a Pin? What does sleep() do? Change the timings.

## CHALLENGE

Can you make it blink faster or slower?

# Wire the external LED

Ask opencode to show how to blink an external LED — it draws the whole circuit, wires included.

## CLICK A WIRE

Every wire is clickable: click one to see where it connects, end to end. The numbered build order and the notes come with the diagram.

## THEN THIS

Ask opencode: “Generate a wiring table that summarizes the connections.”

A wiring table makes it easy to check your board before you power it on.

**Add an external LED — Freenove ESP32-WROVER (FNK0046)**  
 Four steps, two jumper wires — the ESP32 board stays off the breadboard

**Legend:**

- Red line — signal — GPIO 2
- Black line — ground — GND
- Orange dot — board pin
- Yellow line — one connected net (5 holes)
- Blue dot — build order

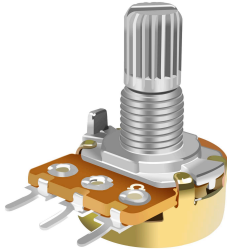
**Steps**

1. Red jumper: board GPIO 2 → breadboard column 2, top row.
2. Resistor 220  $\Omega$ : one leg in column 2, other leg in column 6 (same row).
3. LED: long leg (anode) in column 6, short leg (cathode) in column 7, same row.
4. Black jumper: breadboard column 7, bottom row → board GND (right header, pin 17).

**Never wire an LED straight to 3.3 V or a GPIO without the resistor — it will burn out.**  
 The five holes in one breadboard column are already connected — that is the whole point of a breadboard.  
 GPIO 2 also drives the onboard LED, so both light together. Move the red wire to GPIO 4 for a separate LED.

# Knobs, sticks, and **sensors**

Your robot reads the world through inputs. Here's how two common ones wire up.



Potentiometer. A knob the ESP32 can read

## POTENTIOMETER

- Left pin → GND
- Middle pin (wiper) → ESP32
- Right pin → 3V3



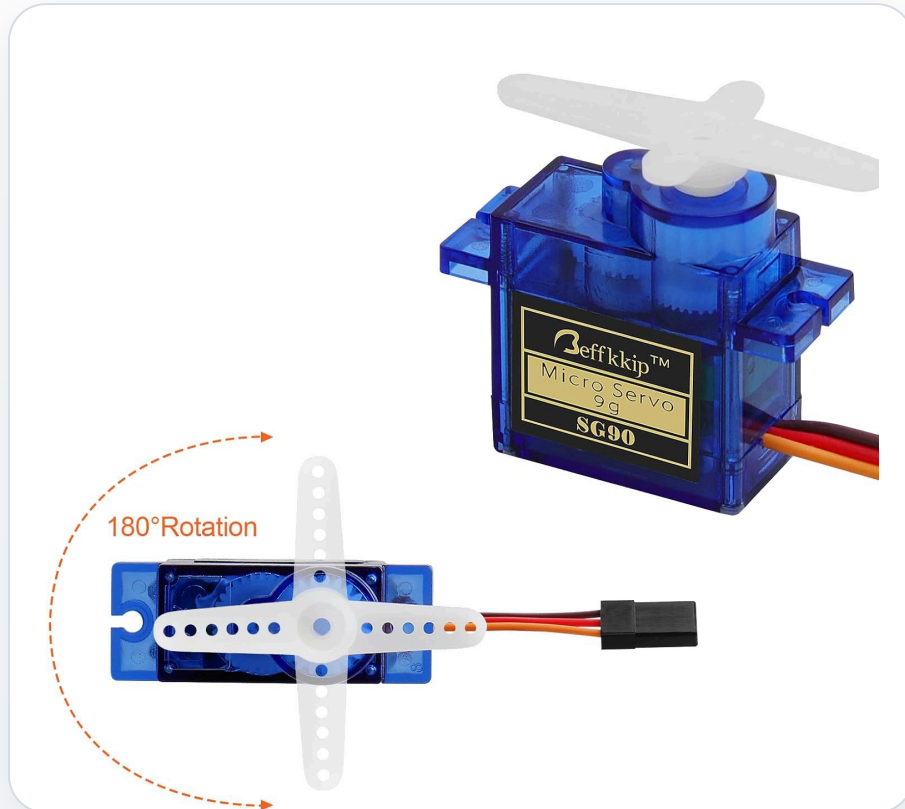
KY-023 joystick. Two axes plus a button

## KY-023 JOYSTICK

- GND → GND
- +5V → 3V3
- VRx (X-axis) → ESP32
- VRy (Y-axis) → ESP32
- SW (button) → ESP32

# Move an arm with a servo

The SG90 micro servo rotates about 180°. Perfect for arms, gates, and grippers.



## SERVO WIRING

- Brown / Black → GND
- Red → 5V
- Orange / Yellow → ESP32

## ON THE ROBOT

Servo 1 and Servo 2 wire up the same way. Match the colors, not the memory.

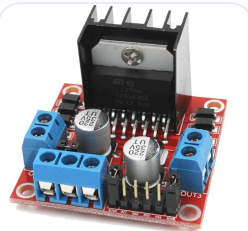
# Spin the wheels with the L298N

The driver is the muscle: the ESP32 sends signals, the L298N sends power to the motors.



## TT Motor

DC gearbox  
motor + wheel



## L298N driver

Dual H-bridge  
motor driver

## L298N WIRING

- 12V → battery pack
- 5V → logic power
- GND → common ground
- IN1, IN2 → ESP32 (left motor)
- IN3, IN4 → ESP32 (right motor)
- OUT1, OUT2 → left motor (red, black)
- OUT3, OUT4 → right motor (red, black)

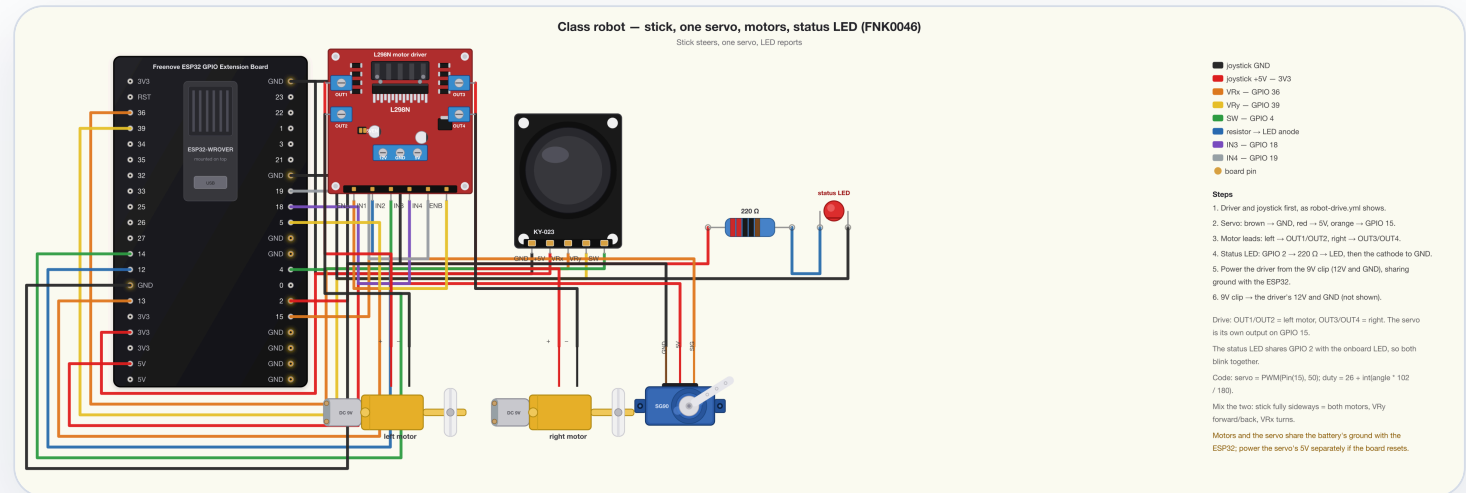
# Robot wiring example

Stick, one servo, two drive motors, one driver, one brain.

## READ IT LIKE A MAP

- Power comes from the battery pack
- Motors run through the L298N driver
- Servo and controls go to GPIO pins

Wire from a diagram, never from memory, and ask AI to explain any wire you don't recognize.



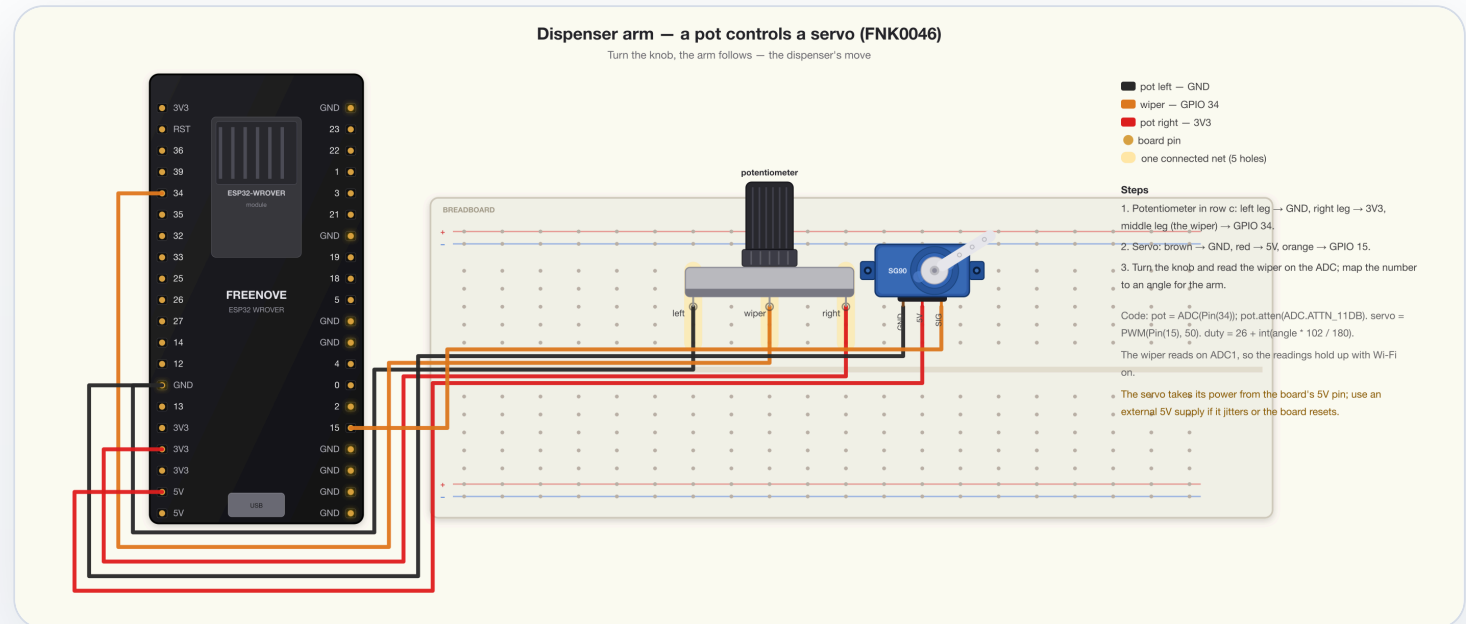
# Dispenser wiring example

A pot and a servo — the smallest complete circuit: one input, one output.

## READ IT LIKE A MAP

- The pot's wiper reads on GPIO 34 — an analogue input
- The servo takes its signal on GPIO 15
- Turn the knob and the arm follows

Build the dispenser's move on its own first, then mount it on the robot.



# Wire your robot **one part at a time**

Mechanical assembly review, then AI-assisted wiring at the makerspace.

## TODAY AT NOLOP

- Mechanical assembly review
- Use AI to wire robot components one by one
- Mount parts and run wires after functional testing

## TIPS

- Do not generate all the code and wiring in one prompt.
- Add components step by step. Test and verify with each iteration.
- Provide detailed model numbers for the parts.

## NAME YOUR PARTS EXACTLY

- DC Gearbox Motor. "TT Motor"
- BOJACK L298N Motor DC Dual H-Bridge Motor Driver
- KY-023 Control Stick
- SG90 9g Micro Servo Motor
- Freenove ESP32 WROOM

# AI-assisted electronics project

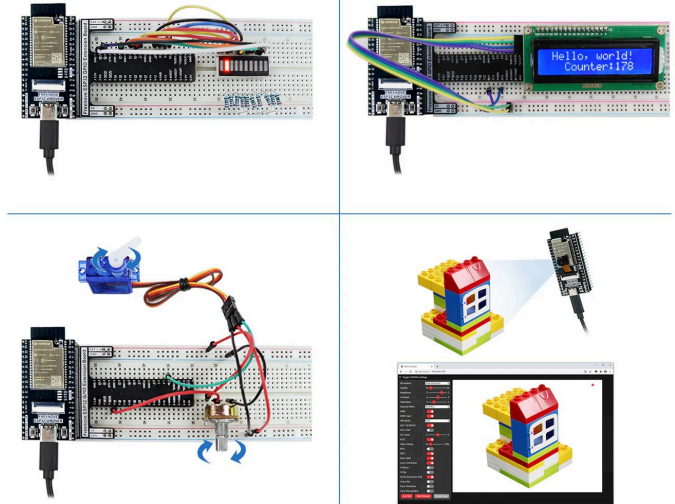
Design and build a fun electronics project using your ESP32 kit. With AI as your coding partner.

## REQUIREMENTS

- At least 1 input (sensor, button, etc.) + 1 output (LED, display, buzzer, etc.)
- AI-generated MicroPython code, tested and refined

## THE PROCESS. ALL IN ONE CHAT

- **Brainstorm:** attach the parts list image, ask AI for project ideas
- **Build:** get wiring instructions and code from AI
- **Test & iterate:** describe problems, share errors, refine with AI's help



Need ideas? The kit tutorial has example projects and code to borrow from.

# What you'll hand in

01

## Your AI conversation

A link to your chat. One continuous conversation, start to finish.

02

## Video demo

A demo of the working project with a voice-over explanation.

03

## Written reflection

A written reflection on the process.

**DEMO** Bring your electronics project to class for a demo.

**GRADING** You're graded on **collaboration and learning**, not coding skills. Keep one chat, iterate often, and ask lots of questions.

# Physical Making **reflection**

A short written reflection on today's build — what your hands knew that your plan didn't.

01

## Catch the surprise

Find the moment the build didn't go the way you pictured it. What did you notice, and what did you do next?

02

## Think on your feet

Point to a decision you made mid-build without stopping to plan it. What were you reacting to?

03

## Look back

Building it again tomorrow: what would you keep exactly as it is, and what would you change — and why?

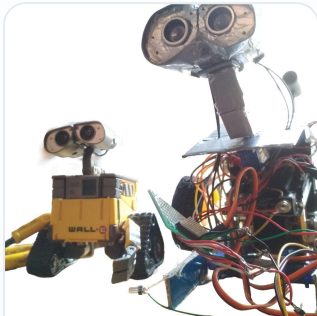
### HOW TO WRITE IT

Ground every answer in a specific moment from the build. A few honest paragraphs beat a polished essay.

Prompts inspired by Donald Schön's reflective practitioner: reflection in action, and reflection on action.

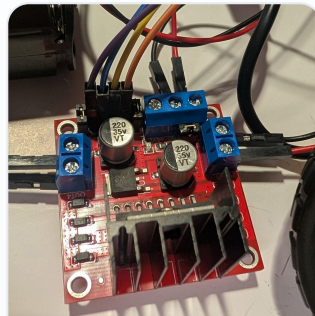
# Robots under construction

Wiring, testing, and mounting. The messy middle of the build.



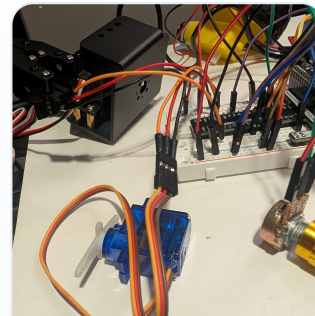
## Wire by wire

The robot takes shape.



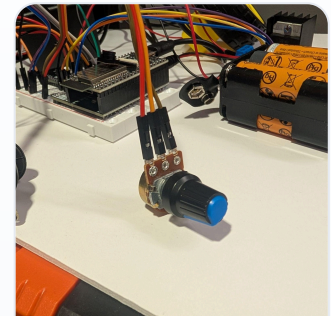
## Motor driver

Powered and wired in place.



## Test rig

Test each part before mounting.



## Controls

Knobs wired and verified.

BEFORE YOU GO

# Questions?

Next week: AI Coding Agents. Connect your microcontroller to a wireless network and build a remote control interface.

## WHERE TO FIND US

### Office hours

[go.tufts.edu/meet\\_tvdv](https://go.tufts.edu/meet_tvdv)

### Email

[thomas.van\\_de\\_velde@tufts.edu](mailto:thomas.van_de_velde@tufts.edu)

### Course materials

[tuftsmaker.github.io/ENT-164](https://tuftsmaker.github.io/ENT-164)